Kontrol Motor DC
Over-The-Air (OTA) Programming Menggunakan ESP32

Dosen Pengampu: Prof. Dr. Basuki Rahmat, S.Si., M.T.

Disusun Oleh:

Nama : Sanafa Mawaddah R. Z

NPM : 24081010153

PROGRAM STUDI INFORMATIKA FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

2026

A. Tujuan Modul

1. Implementasi IoT dan Kontrol Motor: Mahasiswa mampu mengendalikan kecepatan dan arah Motor DC secara nirkabel menggunakan platform Blynk.
2. Over-The-Air (OTA) Programming: Mahasiswa mampu merancang sistem yang dapat menerima pembaruan *firmware* (kode program) secara nirkabel melalui jaringan WiFi tanpa perlu menghubungkan kabel USB ke komputer.
3. Multitasking Non-Blocking: Mahasiswa memahami dan mampu mengimplementasikan fungsi millis() sebagai pengganti delay() untuk menjalankan indikator LED (fisik dan virtual) tanpa menghentikan proses komunikasi data Blynk dan OTA.

B. Skema Rangkaian

Motor Driver (L298N/L293D):

- IN1 dihubungkan ke Pin 27 ESP32.
- IN2 dihubungkan ke Pin 26 ESP32.
- ENA (PWM) dihubungkan ke Pin 12 ESP32.
- VCC/12V ke sumber tegangan eksternal, OUT1/OUT2 ke Motor DC.
- GND Motor Driver digabung (common ground) dengan GND ESP32.

Indikator Fisik (LED):

- Memanfaatkan LED *Built-in* pada Pin 2 ESP32 (tidak memerlukan komponen tambahan).

C. Konsep & Logika Sistem

Sistem ini menggabungkan tiga arsitektur utama yang berjalan secara bersamaan: Pertama, Komunikasi IoT di mana ESP32 terus bertukar data dengan server Blynk untuk menerima perintah motor dan mengirimkan telemetri (RPM/Duty Cycle). Kedua, Sistem OTA (Over-The-Air) yang membuat ESP32 berstatus *standby* di jaringan lokal dengan nama "Robotika-iMCLab". Jika ada *push* program baru dari Arduino IDE, ESP32 akan menangkapnya secara nirkabel. Ketiga, Sistem Heartbeat (Kedip Ganda) yang bertindak sebagai penanda visual bahwa sistem sedang aktif dan tidak *hang*. Karena kontrol Blynk dan OTA membutuhkan koneksi terus-menerus tanpa jeda, sistem *heartbeat* ini mutlak harus menggunakan millis() agar waktu pembacaan jaringan tidak terblokir oleh fungsi delay().

D. Daftar Perintah

- V3 (Button/Switch): Saklar daya utama motor (1 = ON, 0 = OFF).
- V2 (Slider): Pengatur kecepatan motor dengan rentang Duty Cycle (0 - 255).

- V1 (Gauge/Value Display): Menampilkan nilai Duty Cycle aktual yang sedang aktif.
- V0 (Gauge/Value Display): Menampilkan simulasi kecepatan motor dalam satuan RPM (0 - 13000).
- V4 (LED Widget): Indikator virtual di aplikasi Blynk yang akan berkedip setiap 1 detik sebagai tanda status *standby*, dan menyala penuh saat proses *update* OTA berlangsung.

E. Source Code dan Penjelasan Logika Program

```
#define BLYNK_TEMPLATE_ID ""
#define BLYNK_TEMPLATE_NAME ""
#define BLYNK_AUTH_TOKEN ""

#define BLYNK_PRINT Serial

// --- PENGATURAN WIFI ---
char ssid[] = "";
char pass[] = "";
// -----------------------

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

// ========================================
// LIBRARY UNTUK OTA (OVER-THE-AIR)
#include <ESPmDNS.h>
#include <WiFiUdp.h>
#include <ArduinoOTA.h>
// ========================================

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// --- PENGATURAN INDIKATOR OTA (KEDIP GANDA) ---
#define LED_BUILTIN 2
unsigned long previousMillis = 0;
const long interval = 1000; // Berkedip tiap 1 detik
int ledState = LOW;
int blynkLedState = 0; // Status untuk LED di aplikasi Blynk
```

```
// ------------------------------------------------

const int freq = 30000;
const int resolution = 8;
int dutyCycle = 0;
bool isMotorRunning = false;

void applyMotor() {
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);

    Blynk.virtualWrite(V1, dutyCycle);

    int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
    Blynk.virtualWrite(V0, simulasiRPM);

  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);

    Blynk.virtualWrite(V1, 0);
    Blynk.virtualWrite(V0, 0);
  }
}

BLYNK_WRITE(V3) {
  int buttonState = param.asInt();
  if (buttonState == 1) {
    isMotorRunning = true;
    if (dutyCycle == 0) dutyCycle = 150;
    Serial.println("BLYNK: Motor dinyalakan!");
  } else {
    isMotorRunning = false;
    Serial.println("BLYNK: Motor dimatikan!");
  }
  applyMotor();
}

BLYNK_WRITE(V2) {
```

```
  dutyCycle = param.asInt();
  Serial.print("BLYNK: Slider diubah ke kecepatan: ");
  Serial.println(dutyCycle);

  if (isMotorRunning) {
    applyMotor();
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(LED_BUILTIN, OUTPUT);

  ledcAttach(enable1Pin, freq, resolution);

  isMotorRunning = false;
  applyMotor();

  Serial.println("\nMenghubungkan ESP32 ke WiFi dan Server
Blynk...");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Berhasil Terhubung ke Blynk Cloud!");

  // ========================================
  // PENGATURAN AWAL SISTEM OTA
  // ========================================
  ArduinoOTA.setHostname("Robotika-iMCLab");

  ArduinoOTA.onStart([]() {
    Serial.println("\n[OTA] Memulai proses update OTA...");
    digitalWrite(LED_BUILTIN, HIGH);
    Blynk.virtualWrite(V4, 255); // LED Blynk menyala terang
saat update
  });

  ArduinoOTA.onEnd([]() {
    Serial.println("\n[OTA] Update OTA Selesai!");
    digitalWrite(LED_BUILTIN, LOW);
    Blynk.virtualWrite(V4, 0);
```

```
  });

  ArduinoOTA.onProgress([](unsigned int progress, unsigned int
total) {
    Serial.printf("[OTA] Progress: %u%%\r", (progress / (total /
100)));
  });

  ArduinoOTA.onError([](ota_error_t error) {
    Serial.printf("[OTA] Error[%u]: ", error);
  });

  ArduinoOTA.begin();
  Serial.println("[OTA] Sistem OTA Aktif dan Standby!");
  // ========================================
}

void loop() {
  Blynk.run();
  ArduinoOTA.handle();

  // --- LOGIKA KEDIP GANDA (FISIK & BLYNK) SEBAGAI TANDA
STANDBY ---
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    // 1. Kedipkan LED Fisik di Robot
    ledState = (ledState == LOW) ? HIGH : LOW;
    digitalWrite(LED_BUILTIN, ledState);

    // 2. Kedipkan LED Virtual di Layar HP (Blynk)
    blynkLedState = (blynkLedState == 0) ? 255 : 0;
    Blynk.virtualWrite(V4, blynkLedState);
  }
}
```

Penjelasan program

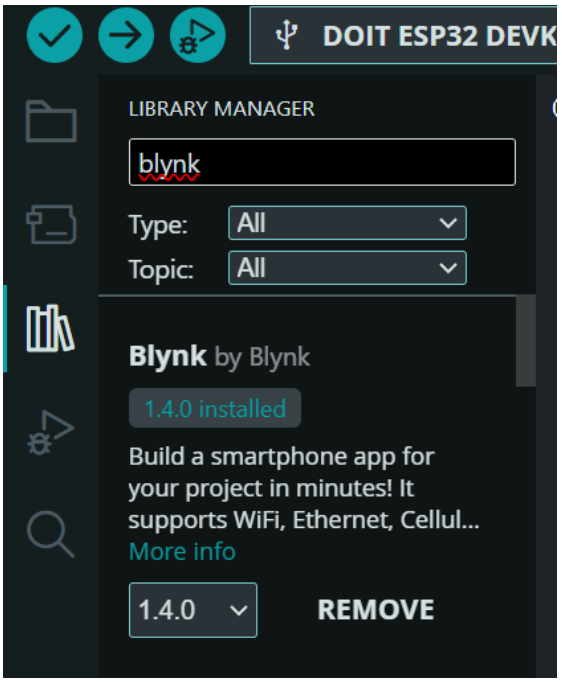
1. Konfigurasi OTA: Pada fungsi setup(), library ArduinoOTA diinisialisasi. Hostname diatur agar port jaringan mudah dikenali di Arduino IDE. Program mendaftarkan fungsi *callback* (seperti onStart, onEnd, dan onProgress) untuk

mengatur perilaku perangkat saat menerima *file flash*, termasuk memaksa LED fisik dan virtual (V4) menyala terang penuh (255) selama proses unggah berlangsung.

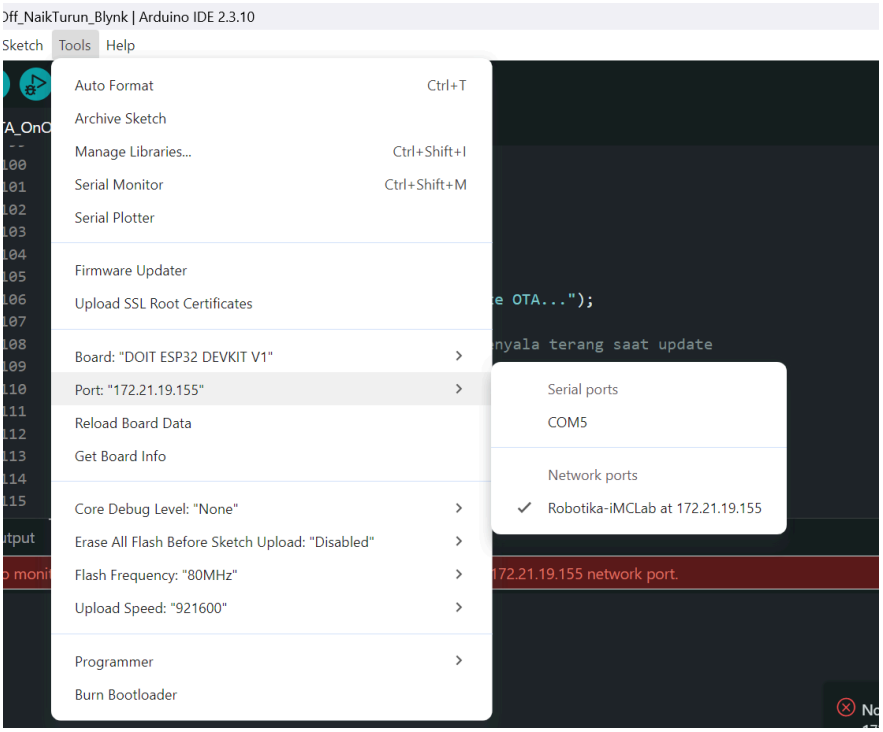
2. Polling Rutin (Loop): Fungsi loop() didesain sangat minimalis dan non-interuptif. Blynk.run() bertugas menangani paket masuk/keluar ke server Blynk, sementara ArduinoOTA.handle() bertugas memantau ketersediaan file *update* di jaringan lokal.
3. Multitasking LED dengan Millis: Logika pengkondisian currentMillis - previousMillis >= interval memungkinkan mikrokontroler mengecek apakah waktu 1 detik telah berlalu. Jika ya, variabel status LED dibalik nilainya (dari redup ke terang atau sebaliknya). Karena metode ini hanya mengecek selisih waktu, sisa waktu pada siklus loop dapat digunakan oleh Blynk dan OTA untuk bekerja di belakang layar.

F. Hasil Pengujian & Analisis

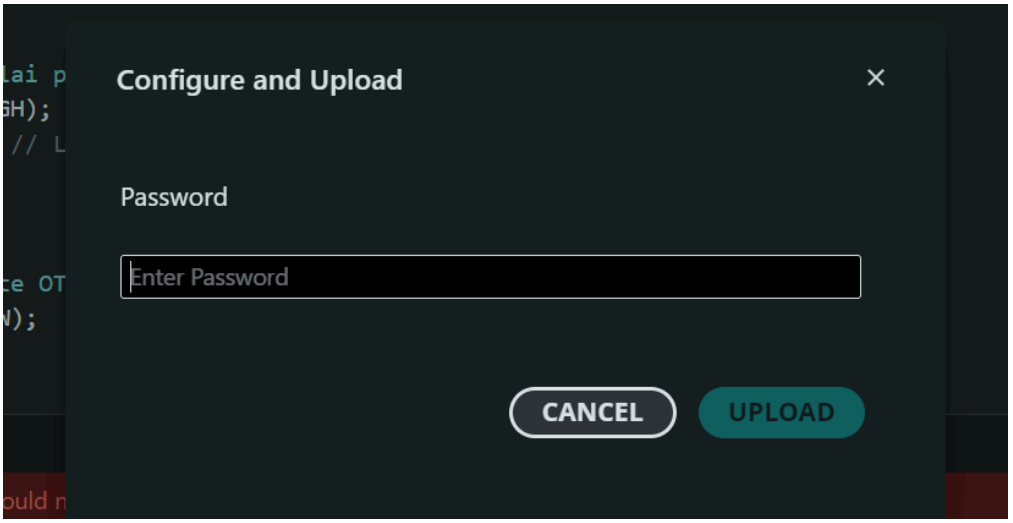
1. Proses Kompilasi dan Upload Program



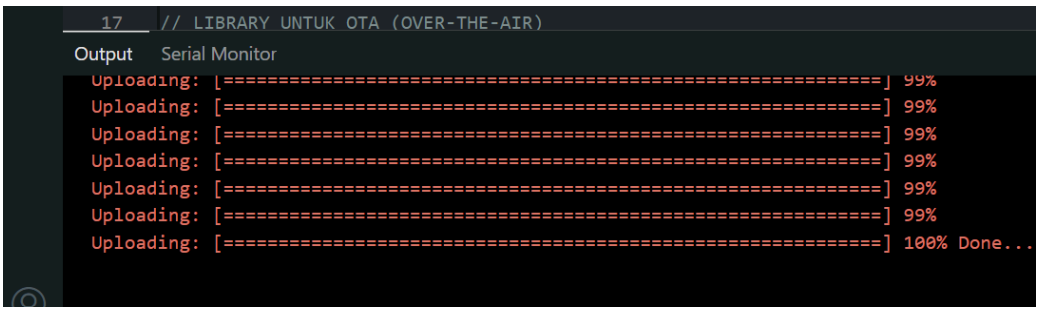
Sebelum memulai, pastikan library utama seperti Blynk telah terinstal melalui Library Manager. Untuk sistem OTA sendiri, library seperti ArduinoOTA, ESPmDNS, dan WiFiUdp sudah menjadi bawaan (built-in) saat board ESP32 diinstal pada Arduino IDE, sehingga siap digunakan.



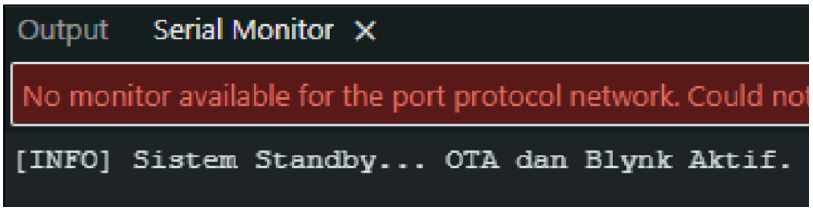
Berbeda dengan modul sebelumnya yang menggunakan kabel USB (COM Port), pada sistem OTA kita memilih *Network ports*. Gambar menunjukkan bahwa ESP32 berhasil memancarkan identitasnya di jaringan lokal dengan *hostname* "Robotika-iMCLab at 172.21.19.155" dan siap menerima injeksi *firmware* secara nirkabel.



Saat tombol Upload ditekan, IDE akan memunculkan pop-up Configure and Upload. Jika pada kode program OTA disetel sebuah password keamanan (untuk mencegah orang asing mengubah program robot), pengguna wajib memasukkan password tersebut di jendela ini sebelum proses transfer dimulai.



Gambar ini memperlihatkan antarmuka *console* saat proses pengiriman *file biner* (program) melalui jaringan WiFi (Over-The-Air). Indikator persentase akan berjalan hingga mencapai tanda 100% Done..., yang mengonfirmasi bahwa mikrokontroler telah berhasil diperbarui tanpa sentuhan kabel fisik.



Terdapat peringatan *"No monitor available for the port protocol network"* pada Serial Monitor. Hal ini normal karena Arduino IDE tidak mendukung pembacaan Serial.print secara langsung lewat jaringan WiFi (OTA). Pembacaan status hanya bisa dilakukan secara fisik via kabel, atau dialihkan menjadi indikator virtual di layar aplikasi Blynk.

2. Analisis

Berdasarkan pengujian yang telah dilakukan, sistem penggabungan antara IoT (Blynk) dan *Over-The-Air* (OTA) Programming pada ESP32 telah berjalan dengan sukses. Penggunaan fitur OTA terbukti sangat revolusioner untuk sistem robotik karena memungkinkan pemeliharaan (*maintenance*) dan pembaruan logika *source code* jarak jauh tanpa perlu membongkar sasis robot untuk mencolokkan kabel USB.

Keberhasilan sistem ini sangat bergantung pada implementasi fungsi millis() sebagai *multitasking non-blocking*. Pembuatan indikator LED berkedip (Heartbeat) menggunakan logika currentMillis - previousMillis memastikan mikrokontroler tidak pernah "berhenti" atau "terjeda" (seperti halnya kelemahan pada fungsi delay()). Dengan hilangnya jeda waktu eksekusi, fungsi utama pembacaan jaringan yaitu Blynk.run() dan pendengar

update jaringan ArduinoOTA.handle() dapat terus berjalan secara paralel di latar belakang tanpa terputus. Hal ini mencegah putusnya koneksi perangkat (*Timeout/Offline*) dari server *cloud* Blynk, sekaligus memastikan perangkat selalu *standby* menerima *file firmware* baru setiap saat.